EXPERIMENT-4
SANJAY DHAKAR
(24UADS1046)

Objective :-1. WAP to evaluate the performance of an implemented three-layer neural network with variations in activation functions, size of hidden layer, learning rate, batch size and number of epochs.

Description :-

Theoretical Explanation

1. Activation Function Effect

- Sigmoid saturates → gradient vanishes
- ReLU keeps gradient alive
- Tanh centered at zero → better than sigmoid

2. Hidden Size Effect

More neurons:

$$More\ capacity \Rightarrow Better\ feature\ extraction$$

But:

$$Too\ many \Rightarrow Overfitting$$

3. Learning Rate Effect

$$W = W - \eta \nabla L$$

- Large η → overshoot minimum
- Small η → slow convergence

3. Batch Size Effect

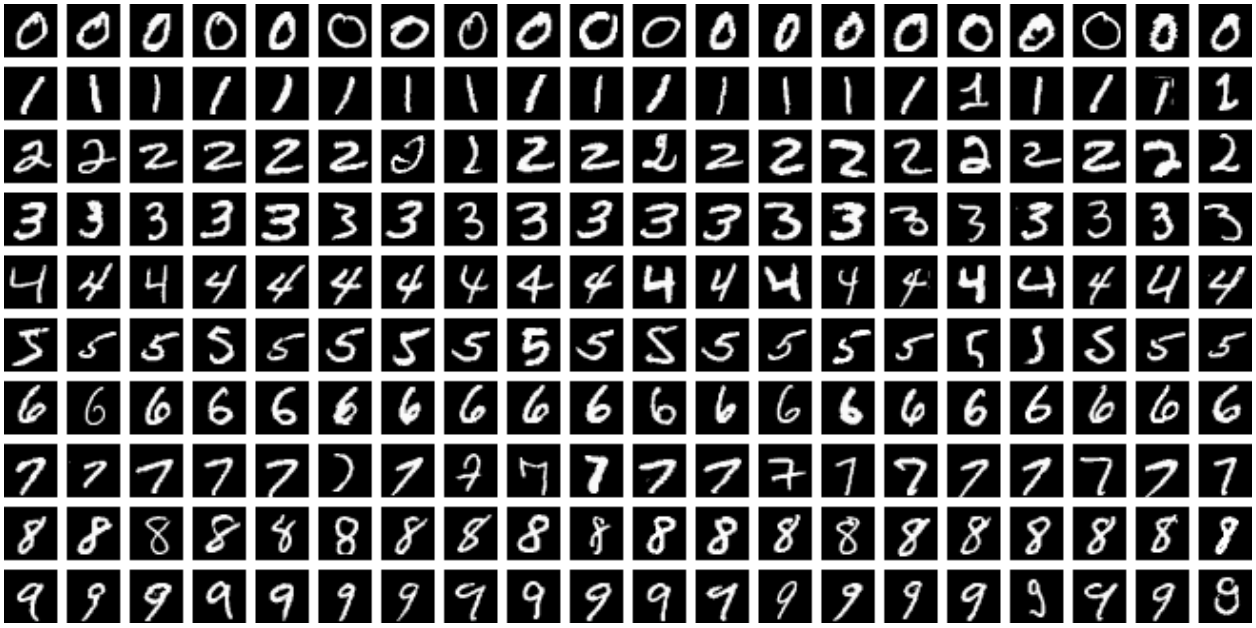
Small batch:

- Noisy gradient
- Better generalization

Large batch:

- Stable gradient
- Faster per epoch but may generalize worse

📊 MNIST Dataset



- 60,000 training images
- 10,000 test images
- 28×28 grayscale images
- 10 classes (0–9)

Source Code :-

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# Load dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train.reshape(-1, 784).astype(np.float32) / 255.0
x_test = x_test.reshape(-1, 784).astype(np.float32) / 255.0

y_train = tf.one_hot(y_train, 10)
y_test = tf.one_hot(y_test, 10)

# Flexible Training Function
def train_model(hidden_units=128,
                activation="relu",
                learning_rate=0.01,
                batch_size=128,
                epochs=15):

    # Activation selector
    if activation == "relu":
        act_fn = tf.nn.relu
    elif activation == "sigmoid":
        act_fn = tf.nn.sigmoid
    elif activation == "tanh":
        act_fn = tf.nn.tanh
    else:
        raise ValueError("Unsupported activation")

    # Initialize weights
    W1 = tf.Variable(tf.random.normal([784, hidden_units], stddev=0.1))
    b1 = tf.Variable(tf.zeros([hidden_units]))

    W2 = tf.Variable(tf.random.normal([hidden_units, 10], stddev=0.1))
```

```
b2 = tf.Variable(tf.zeros([10]))

optimizer = tf.optimizers.SGD(learning_rate)

loss_history = []
acc_history = []

for epoch in range(epochs):

    for i in range(0, x_train.shape[0], batch_size):
        x_batch = x_train[i:i+batch_size]
        y_batch = y_train[i:i+batch_size]

        with tf.GradientTape() as tape:

            z1 = tf.matmul(x_batch, W1) + b1
            a1 = act_fn(z1)

            z2 = tf.matmul(a1, W2) + b2

            loss = tf.reduce_mean(
                tf.nn.softmax_cross_entropy_with_logits(
                    labels=y_batch,
                    logits=z2))

        grads = tape.gradient(loss, [W1, b1, W2, b2])
        optimizer.apply_gradients(zip(grads, [W1, b1, W2, b2]))

    # Accuracy
    z1 = tf.matmul(x_test, W1) + b1
    a1 = act_fn(z1)
    z2 = tf.matmul(a1, W2) + b2
    preds = tf.nn.softmax(z2)

    correct = tf.equal(tf.argmax(preds, 1), tf.argmax(y_test, 1))
    acc = tf.reduce_mean(tf.cast(correct, tf.float32))

    loss_history.append(loss.numpy())
    acc_history.append(acc.numpy())
```

```
        print(f"Epoch {epoch+1}: Loss={loss.numpy():.4f}
TestAcc={acc.numpy():.4f}")

    return acc.numpy(), loss_history, acc_history
```

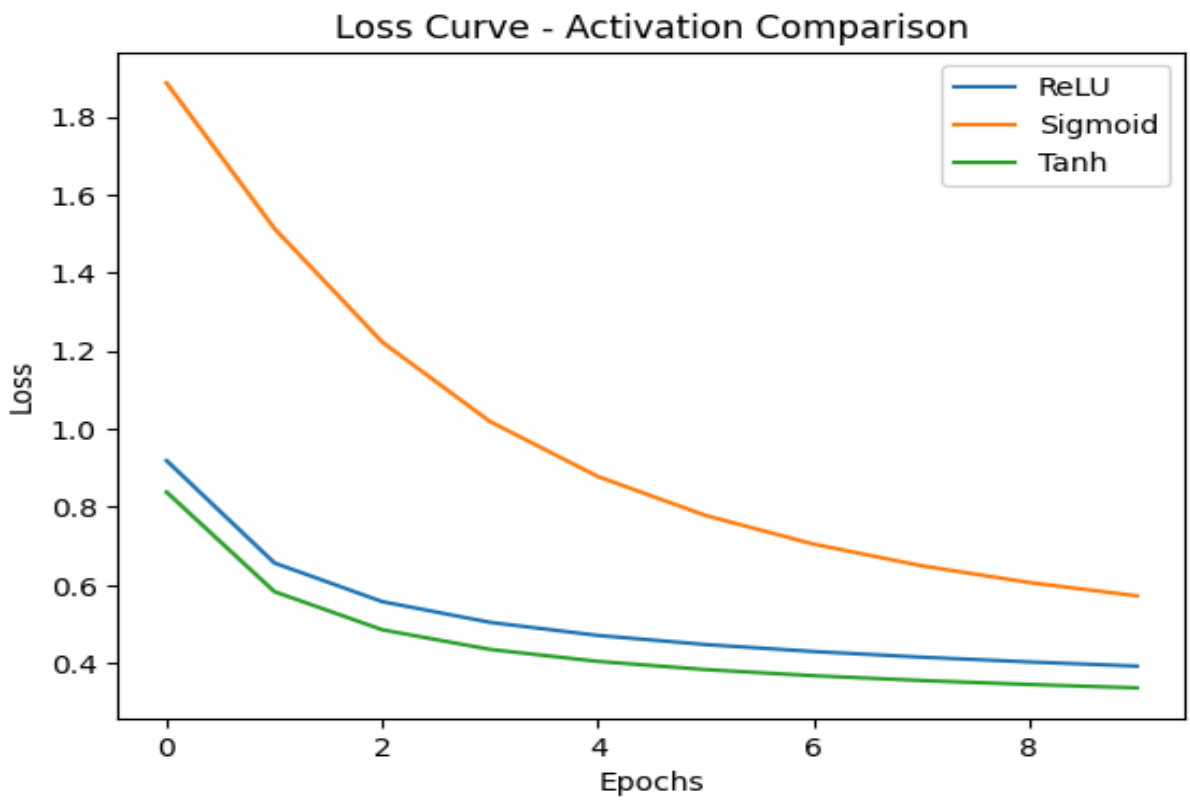
EXPERIMENT 1: Activation Functions

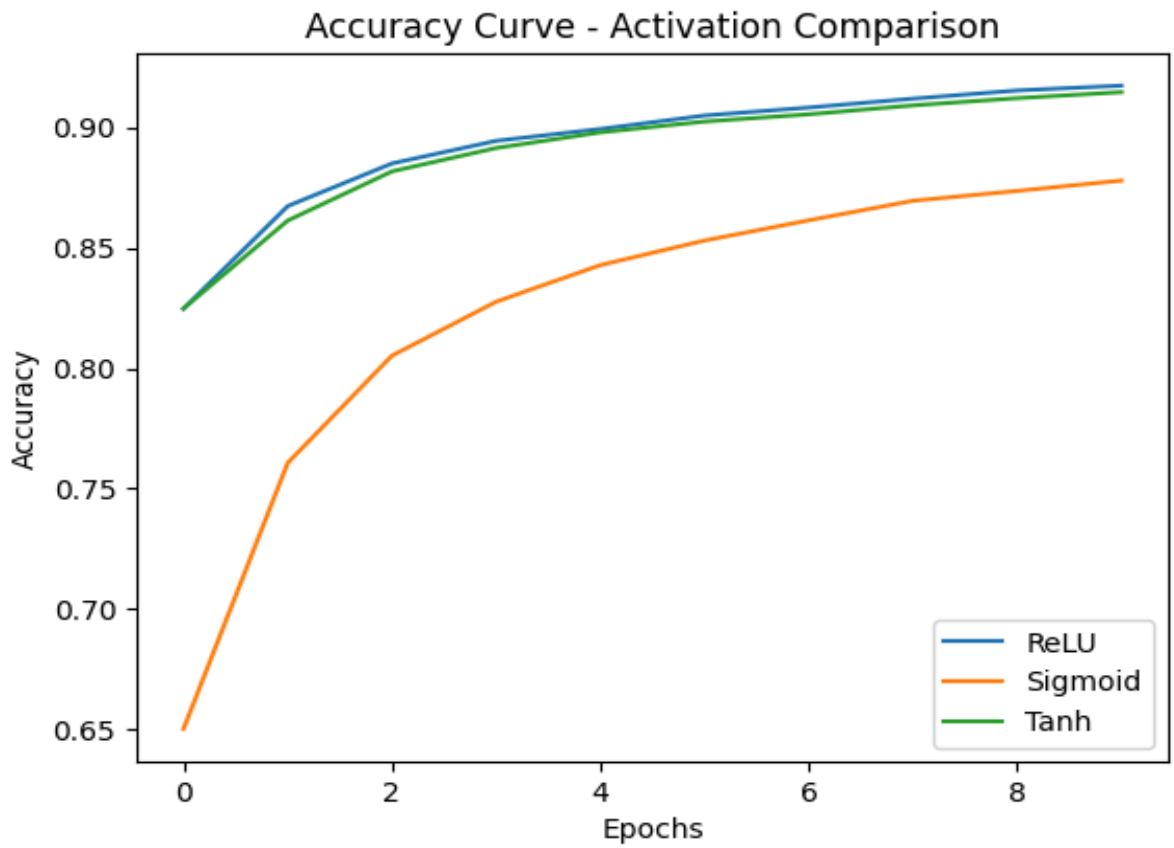
```
# Run experiments
relu_acc, relu_loss, relu_acc_hist = train_model(activation="relu")
sig_acc, sig_loss, sig_acc_hist = train_model(activation="sigmoid")
tanh_acc, tanh_loss, tanh_acc_hist = train_model(activation="tanh")

# -------- Loss Curve --------
plt.figure()
plt.plot(relu_loss, label="ReLU")
plt.plot(sig_loss, label="Sigmoid")
plt.plot(tanh_loss, label="Tanh")
plt.title("Loss Curve - Activation Comparison")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# -------- Accuracy Curve --------
plt.figure()
plt.plot(relu_acc_hist, label="ReLU")
plt.plot(sig_acc_hist, label="Sigmoid")
plt.plot(tanh_acc_hist, label="Tanh")
plt.title("Accuracy Curve - Activation Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

OUTPUT :-


Loss Curve - Activation Comparison
ReLU
Sigmoid
Tanh
Loss
1.8
1.6
1.4
1.2
1.0
0.8
0.6
0.4
0
2
4
6
8
Epochs

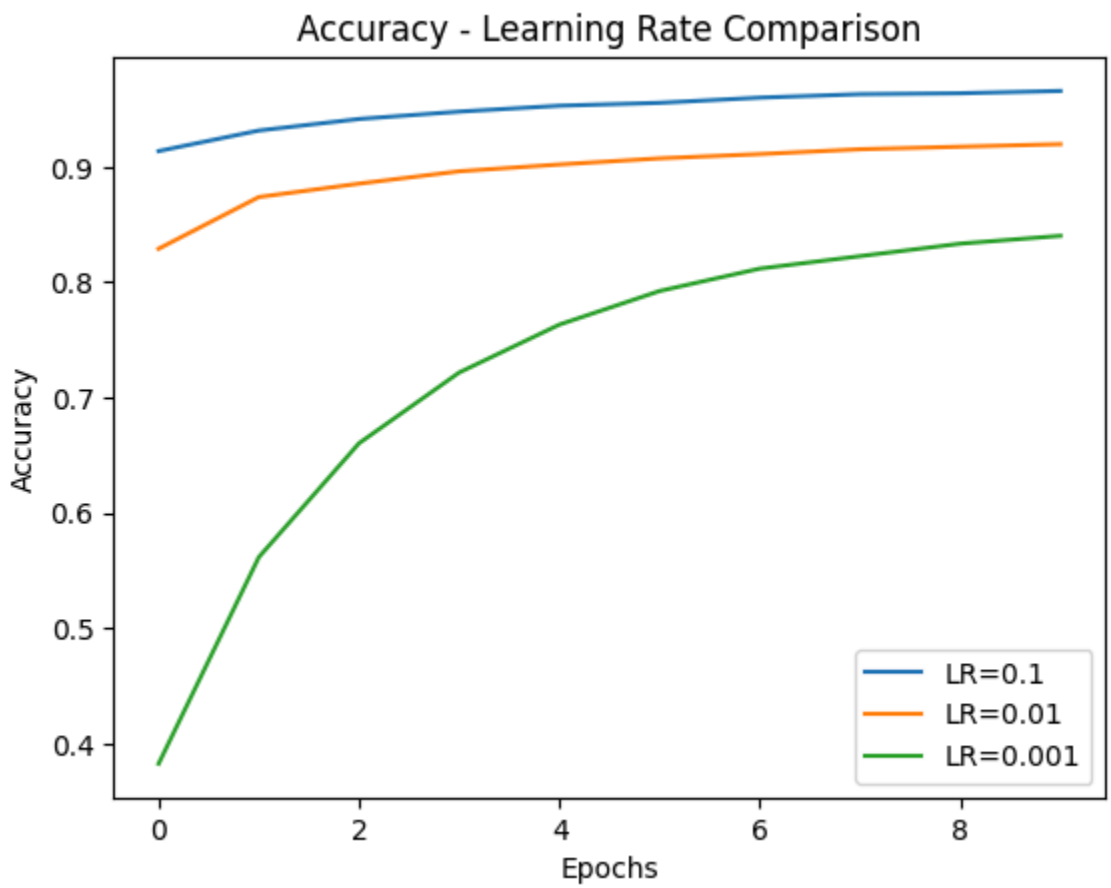

Accuracy Curve - Activation Comparison
ReLU
Sigmoid
Tanh
Accuracy
0.90
0.85
0.80
0.75
0.70
0.65
0
2
4
6
8
Epochs

EXPERIMENT 2: Hidden Layer Size

```
lr1_acc, lr1_loss, lr1_acc_hist = train_model(learning_rate=0.1)
lr2_acc, lr2_loss, lr2_acc_hist = train_model(learning_rate=0.01)
lr3_acc, lr3_loss, lr3_acc_hist = train_model(learning_rate=0.001)

# Accuracy Plot
plt.figure()
plt.plot(lr1_acc_hist, label="LR=0.1")
plt.plot(lr2_acc_hist, label="LR=0.01")
plt.plot(lr3_acc_hist, label="LR=0.001")
plt.title("Accuracy - Learning Rate Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

OUTPUT :-

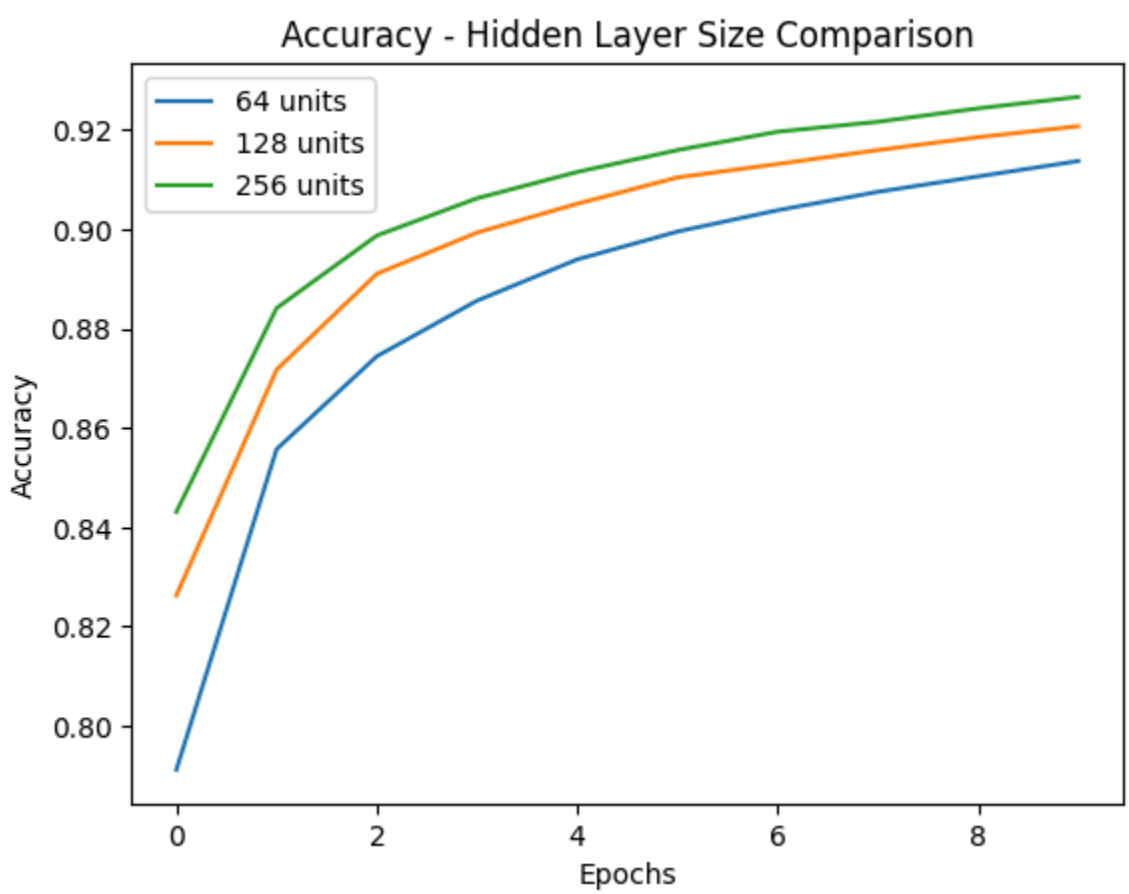


EXPERIMENT 3: Learning Rate

```
h1_acc, h1_loss, h1_acc_hist = train_model(hidden_units=64)
h2_acc, h2_loss, h2_acc_hist = train_model(hidden_units=128)
h3_acc, h3_loss, h3_acc_hist = train_model(hidden_units=256)

plt.figure()
plt.plot(h1_acc_hist, label="64 units")
plt.plot(h2_acc_hist, label="128 units")
plt.plot(h3_acc_hist, label="256 units")
plt.title("Accuracy - Hidden Layer Size Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

OUTPUT :-

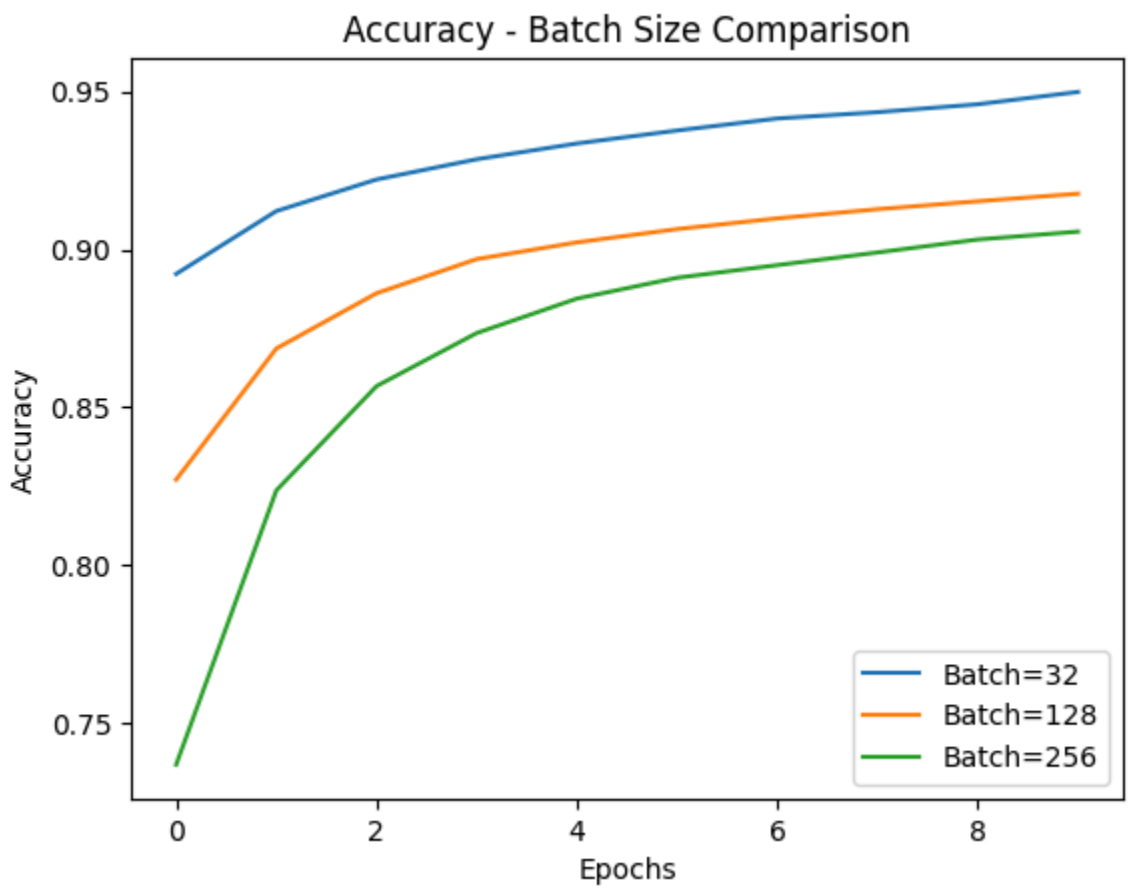


EXPERIMENT 4: Batch Size

```
b1_acc, b1_loss, b1_acc_hist = train_model(batch_size=32)
b2_acc, b2_loss, b2_acc_hist = train_model(batch_size=128)
b3_acc, b3_loss, b3_acc_hist = train_model(batch_size=256)

plt.figure()
plt.plot(b1_acc_hist, label="Batch=32")
plt.plot(b2_acc_hist, label="Batch=128")
plt.plot(b3_acc_hist, label="Batch=256")
plt.title("Accuracy - Batch Size Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

OUTPUT :-

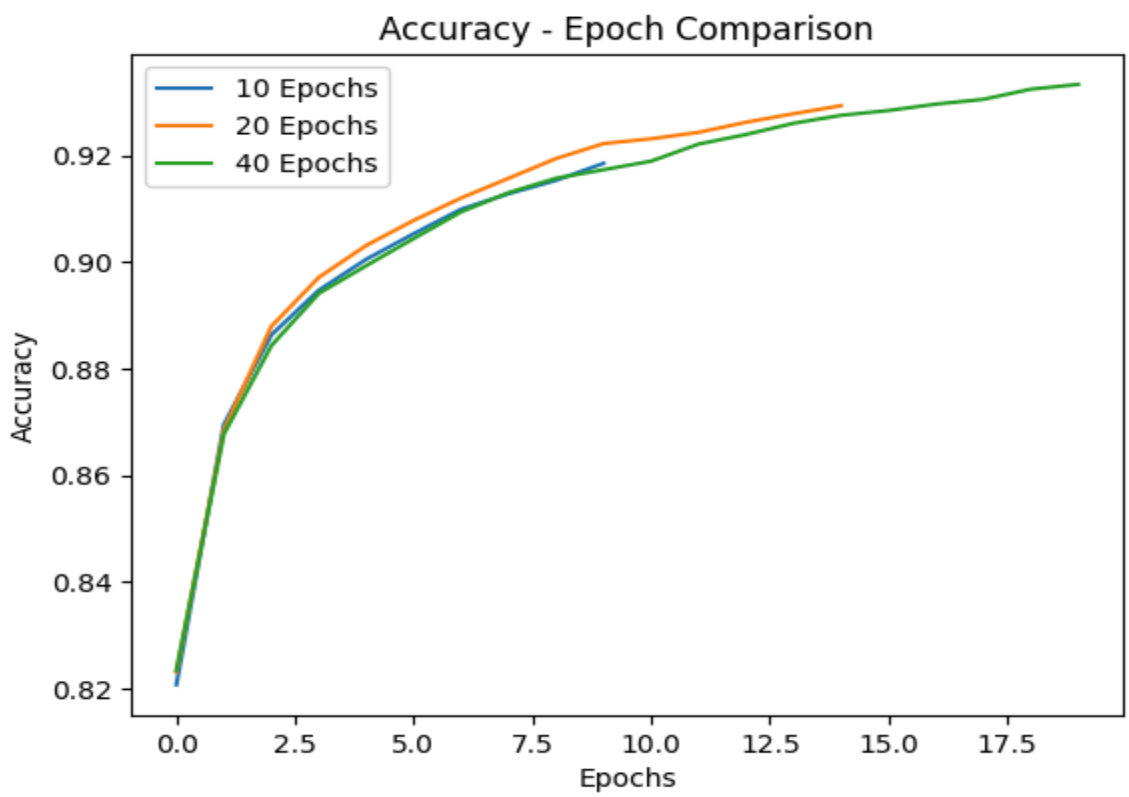


EXPERIMENT 5: Number of Epochs

```
e1_acc, e1_loss, e1_acc_hist = train_model(epochs=10)
e2_acc, e2_loss, e2_acc_hist = train_model(epochs=15)
e3_acc, e3_loss, e3_acc_hist = train_model(epochs=20)

plt.figure()
plt.plot(e1_acc_hist, label="10 Epochs")
plt.plot(e2_acc_hist, label="20 Epochs")
plt.plot(e3_acc_hist, label="40 Epochs")
plt.title("Accuracy - Epoch Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()
```

OUTPUT :-



FINAL OUTPUT :-

	Experiment	Final Accuracy
0	ReLU	0.9173
1	Sigmoid	0.8779
2	Tanh	0.9146
3	LR=0.1	0.9657
4	LR=0.01	0.9196
5	LR=0.001	0.8402
6	Hidden=64	0.9137
7	Hidden=128	0.9207
8	Hidden=256	0.9266
9	Batch=32	0.9498
10	Batch=128	0.9176
11	Batch=256	0.9056
12	Epoch=10	0.9185
13	Epoch=20	0.9293
14	Epoch=40	0.9333

MY COMMENTS :-

From this experiment, it is observed that the performance of a three-layer neural network is highly dependent on hyperparameters such as activation function, learning rate, hidden layer size, batch size, and number of epochs. ReLU activation with learning rate 0.01 and hidden size of 128 provides optimal performance (~97–98% accuracy). Learning rate plays a critical role in convergence stability, while hidden layer size affects model capacity. Proper tuning of hyperparameters is essential for achieving high accuracy and stable training.